

RENSSELAER MECHATRONICS

Blinking the LED

Lab Objectives

- Verify Arduino software package for Simulink using a digital output to light a LED
- Communicate with the target board (Arduino) using external mode by changing the brightness of an LED with PWM
- Use overrun detection to determine how fast you can run your code

Table of Contents

Lab Objectives	1
Part 1: Arduino Toolbox Installation for Simulink.....	2
Objective:	2
Simulink Arduino Library Installation:.....	2
How do I find the COM port for my Arduino Board:.....	Error! Bookmark not defined.
Part 2: Communicating with External Mode.....	6
Objective	6
Simulink.....	6
Deploy to Hardware:.....	8
External Mode – communicating with the hardware	9
External Mode:.....	9
External Mode.....	10
Stopping the Simulation:.....	11
Debugging – cannot connect to device or cannot download code	Error! Bookmark not defined.
Checkpoint:	11
Part 3 (Optional): Evaluating code execution speed – how fast can I run my code	13
Objective:	13
Simulink:.....	13
Part 4: Troubleshooting	16
An error appears when trying to run the file similar to “build procedure aborted”, “make command returned an error...”	16
Error appears when trying to open the Simulink file:.....	16
Simulink Support for Arduino Hardware does not appear in the Simulink library browser:.....	17

Device is not recognized in device manager when it is plugged in.....	17
File is not compiling, a long list of errors appears.	17
In addition to the above, make sure that MATLAB working directory is not Windows/system32 or some other system folder that requires admin rights to edit. To be safe, set the working folder to your documents/MATLAB.....	17
Red block in Simulink diagram:.....	17
Error about board not connected to computer	18
Error Dialog. Error using open... Simulink diagram file name has parenthesis in it.....	20

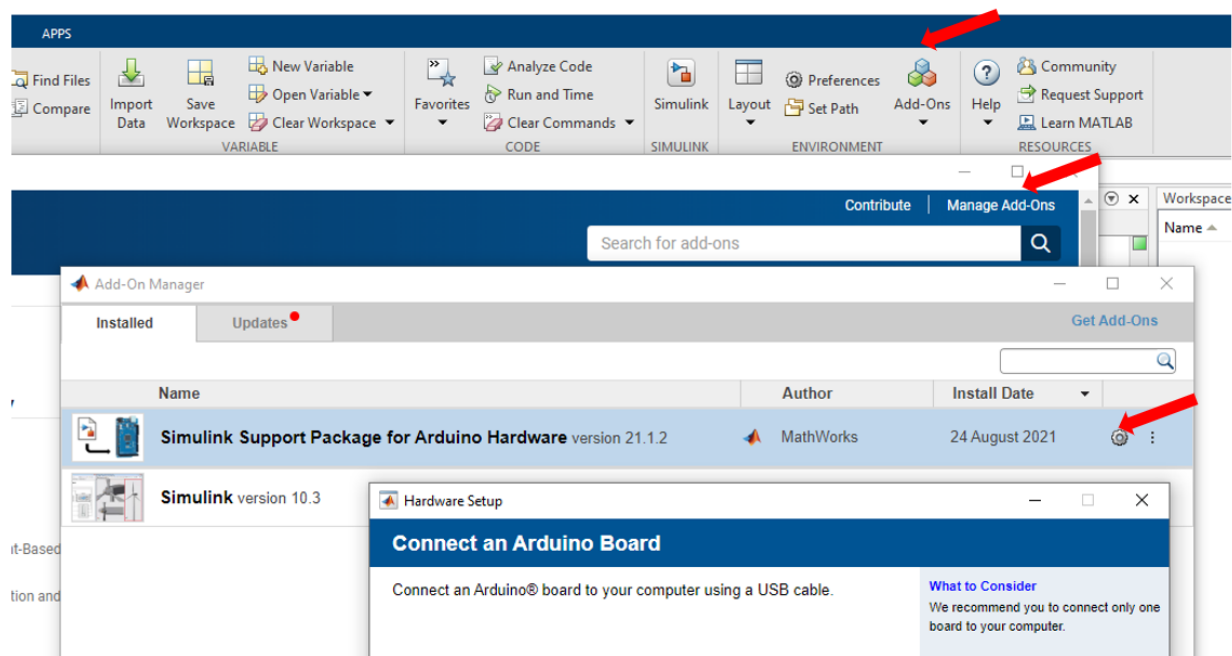
Part 1: Arduino Toolbox Installation for Simulink

Objective:

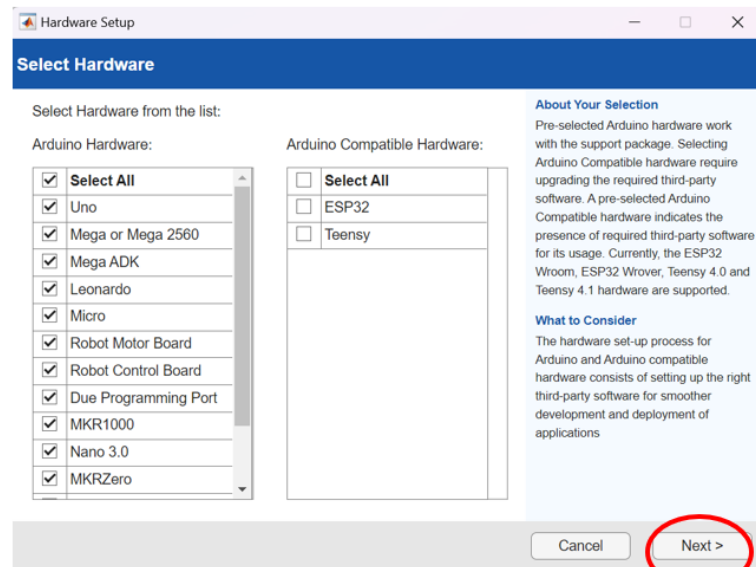
- Install and verify Arduino software package for Simulink using a digital output to light a LED with MATLAB/Simulink 2024a or later.

Verify Simulink Arduino Library Installation:

- Open MATLAB 2024a. From the MATLAB window select “Add-Ons”, “Manage Add-Ons”, and click the gear icon for the Simulink Support Package for Arduino Hardware.



- Leave the default boards selected and select “Next” for the following 3 screens:



The screenshot shows the 'Hardware Setup' window with the 'Select Hardware' tab. It features two lists of hardware options. The 'Arduino Hardware' list has all items checked, including 'Select All', 'Uno', 'Mega or Mega 2560', 'Mega ADK', 'Leonardo', 'Micro', 'Robot Motor Board', 'Robot Control Board', 'Due Programming Port', 'MKR1000', 'Nano 3.0', and 'MKRZero'. The 'Arduino Compatible Hardware' list has 'Select All', 'ESP32', and 'Teensy' unchecked. On the right, there is explanatory text under 'About Your Selection' and 'What to Consider'. At the bottom right, the 'Next >' button is circled in red.

Hardware Setup

Select Hardware

Select Hardware from the list:

Arduino Hardware:

☒	Select All
☒	Uno
☒	Mega or Mega 2560
☒	Mega ADK
☒	Leonardo
☒	Micro
☒	Robot Motor Board
☒	Robot Control Board
☒	Due Programming Port
☒	MKR1000
☒	Nano 3.0
☒	MKRZero

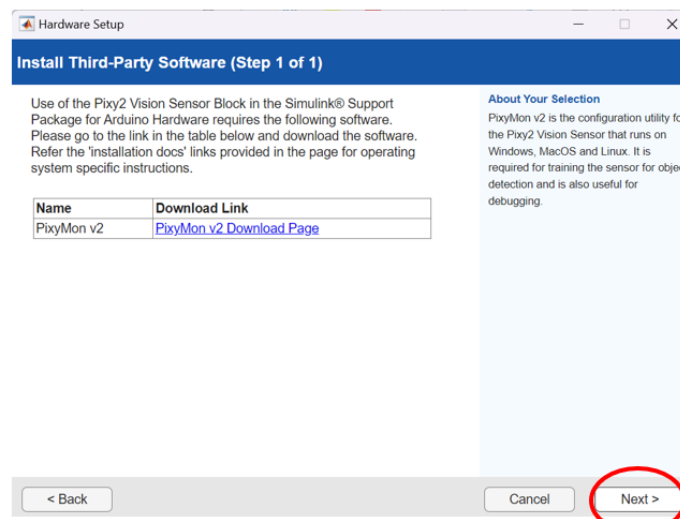
Arduino Compatible Hardware:

☐	Select All
☐	ESP32
☐	Teensy

About Your Selection
Pre-selected Arduino hardware work with the support package. Selecting Arduino Compatible hardware require upgrading the required third-party software. A pre-selected Arduino Compatible hardware indicates the presence of required third-party software for its usage. Currently, the ESP32 Wroom, ESP32 Wrover, Teensy 4.0 and Teensy 4.1 hardware are supported.

What to Consider
The hardware set-up process for Arduino and Arduino compatible hardware consists of setting up the right third-party software for smoother development and deployment of applications

Cancel Next >



The screenshot shows the 'Hardware Setup' window with the 'Install Third-Party Software (Step 1 of 1)' tab. It contains instructions on using the Pixy2 Vision Sensor Block and a table with download links for PixyMon v2. On the right, there is explanatory text under 'About Your Selection'. At the bottom right, the 'Next >' button is circled in red.

Hardware Setup

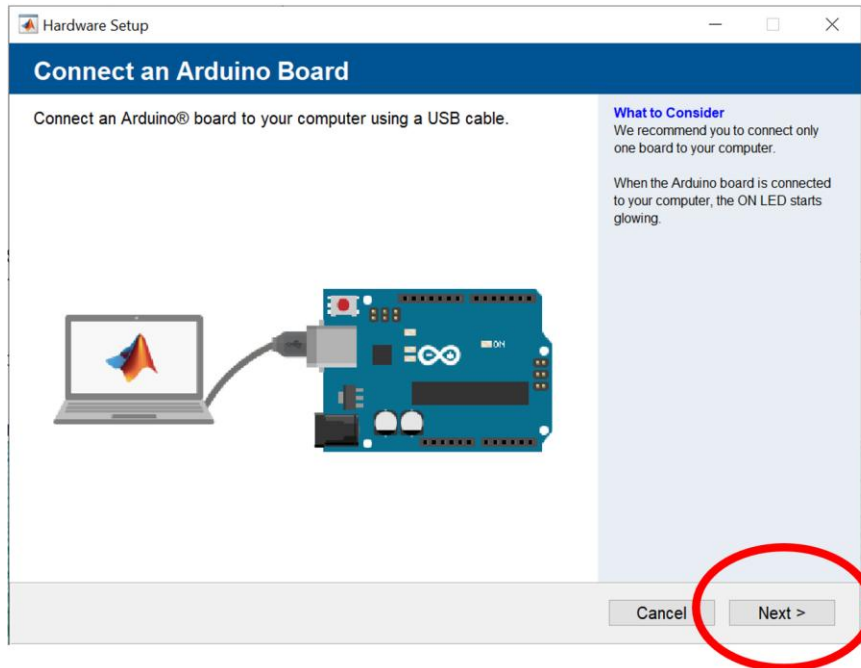
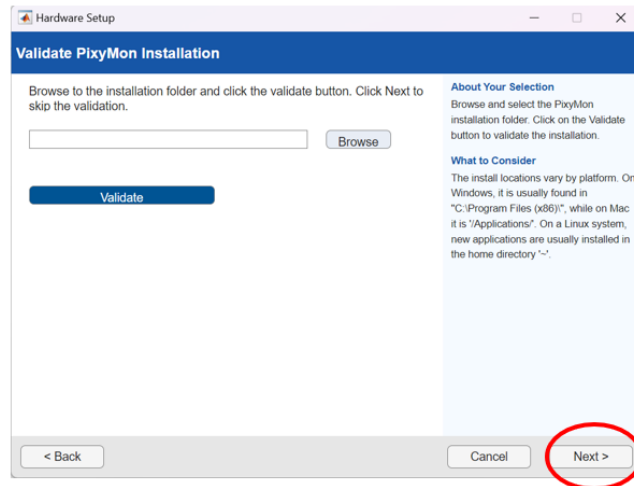
Install Third-Party Software (Step 1 of 1)

Use of the Pixy2 Vision Sensor Block in the Simulink® Support Package for Arduino Hardware requires the following software. Please go to the link in the table below and download the software. Refer the 'installation docs' links provided in the page for operating system specific instructions.

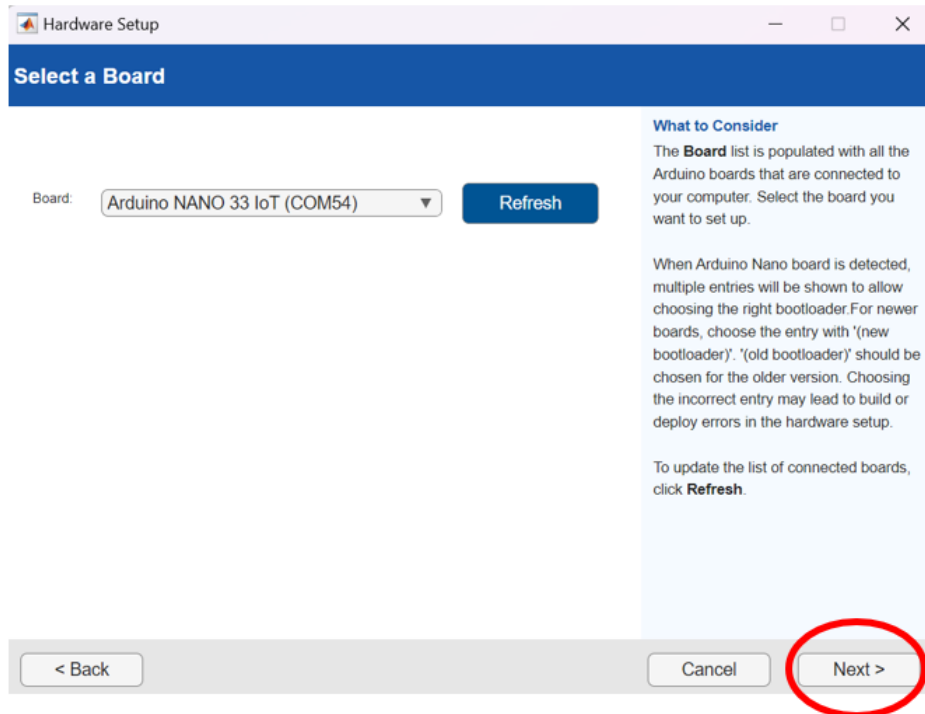
Name	Download Link
PixyMon v2	PixyMon v2 Download Page

About Your Selection
PixyMon v2 is the configuration utility for the Pixy2 Vision Sensor that runs on Windows, MacOS and Linux. It is required for training the sensor for object detection and is also useful for debugging.

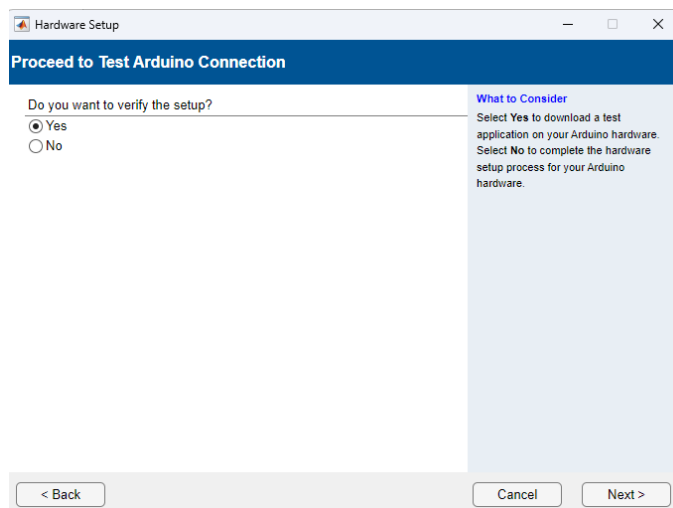
< Back Cancel Next >



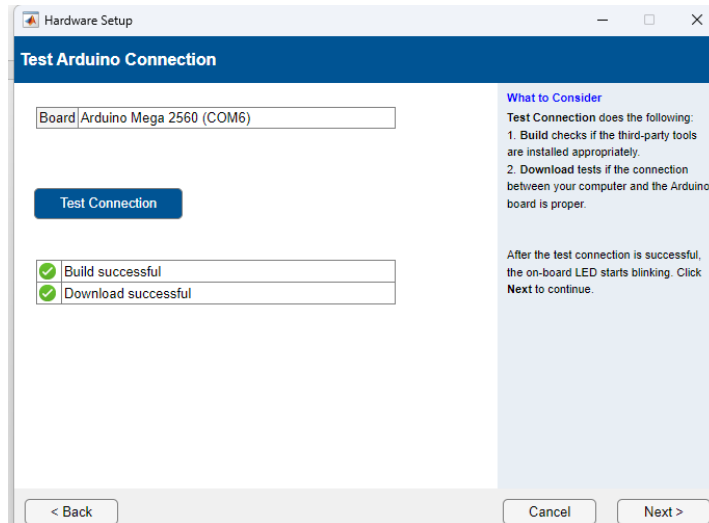
Make sure your board is plugged in and select Next. It will take a couple minutes before getting to the next screen. The next screen should automatically detect your board in which case you can select "Next".



Click Next to verify the setup.



Allow the Java. Verify it builds and downloads:



You are now ready to use your hardware.

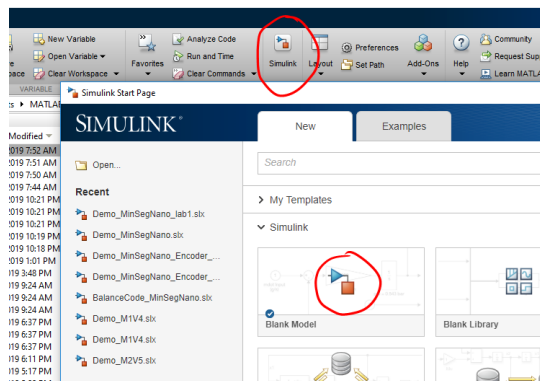
Part 2: Communicating with External Mode

Objective

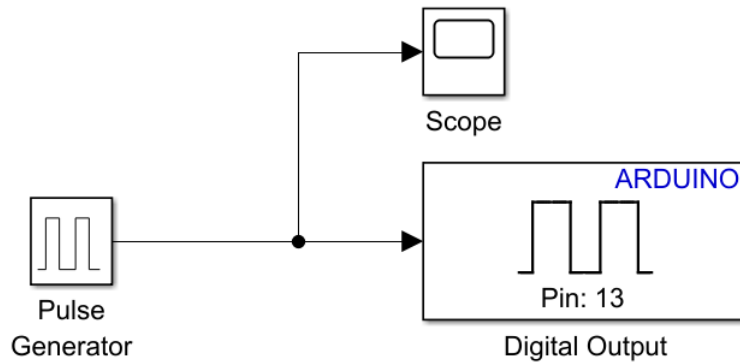
- Communicate with the target board (Arduino) using external mode to turn LED 13 on and off.

Simulink

- 1) Open MATLAB then open a new Simulink model



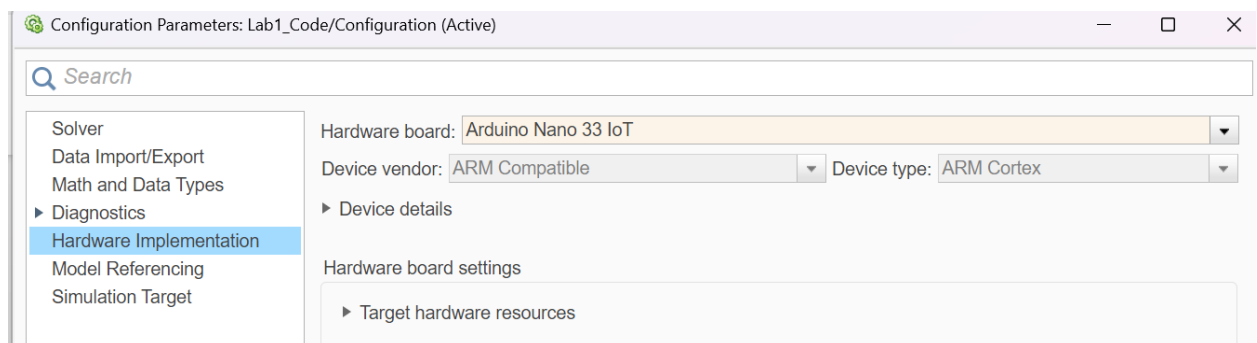
Build the following diagram by following the steps below:



Find and assemble the required blocks.

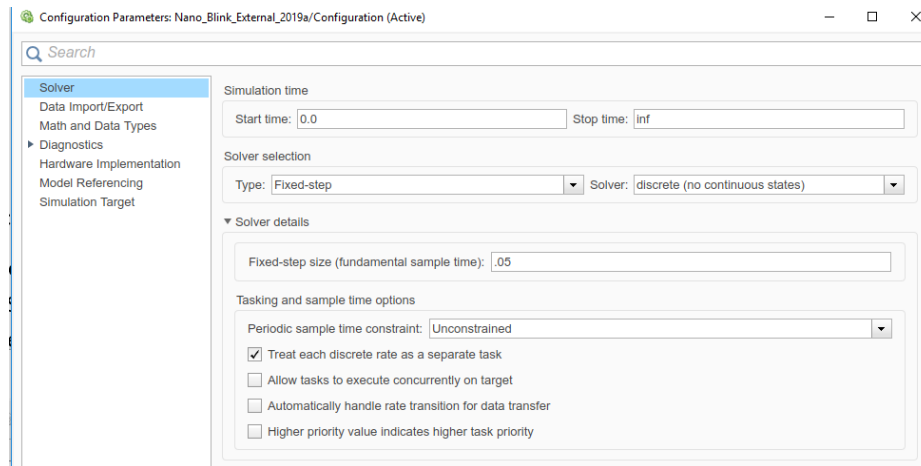
- Pulse Generator is under View->Library Browser->Simulink->Sources
Alternatively click anywhere in your Simulink diagram and start typing “pulse generator”, options will show up as you type, select the one you want
 - Digital Output is under View->Library Browser->Simulink Support Package for Arduino Hardware->Common
Alternatively click anywhere in your Simulink diagram and start typing “digital output”, options will show up as you type, select the one you want
 - Follow the same process for the scope
- 2) Change the output pin to 13, which is connected to the onboard LED. This is done by double clicking on the Digital Output block.
 - 3) Double click on the Pulse Generator to change the amplitude to 1, the Period to 2, and the pulse width to 50%.

- Right click on the background of the Simulink window and select ‘Model Configuration Parameters’, or  in the top of the Simulink window.
- Under ‘Hardware Implementation’, ‘Hardware Board’ Select ‘Arduino nano 33 IoT’, click ‘Apply’ then ‘OK’.



- go to ‘Solver’ on the left hand menu and type ‘inf’ into the ‘Stop time’ spot so that the model will keep running. Expand “Solver details” and make sure the solver options are set to “Fixed-step” and “discrete (no continuous states)”. The “Fixed-step size” should be 0.05. This is how

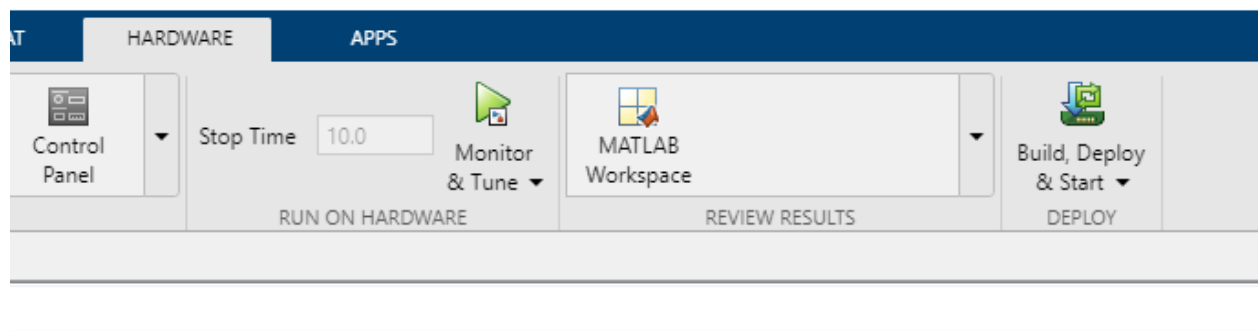
fast in seconds the control loop will execute. In this case, every 50 milliseconds it will execute the Simulink code.



Deploy to Hardware (Normal mode):

Compile and download the code to the board:

- Under the hardware tab select the Build, Deploy and Start button



This will download the code to the board and **will not interact or connect with it**. When “Deploy to Hardware” is used to download to the board no information is passed between it and your computer unless your program specifically instructs the board to do so.

At this point the onboard LED connected to pin 13 should start blinking on and off at every second. If you clock the scope nothing will be displayed because the board is not sending any data to the computer.

At the bottom of the Simulink diagram click ‘View diagnostics’ to view how much memory and program space this program uses:

```
Diagnostic Viewer

▼ Top Model Build

### Starting build procedure for: Lab1_Code
-----
ARM Memory Usage
-----
device - ARM Cortex-M0+

Program: 18496 bytes (7.1% Full)
(.text)

Data: 3588 bytes (10.9% Full)
(.data + .bss)
### Deploying code on Arduino Nano 33 IoT on port COM54
### Deployed code to target successfully
### Successful completion of build procedure for: Lab1_Code

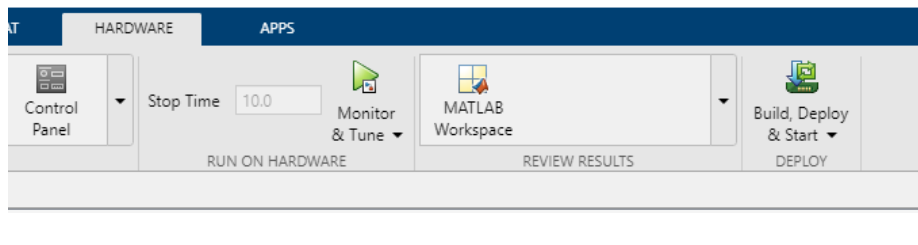
Build process completed successfully
```

Monitor and Tune/External Mode – communicating with the hardware

External mode established a data connection with the system so you can view data or interactively change parameters and see the result on the hardware in real time.

External Mode:

- 1.) External mode enabled by pressing the green “Monitor & Tune” button to download the code. When the green play button is used the hardware will create a connection with your computer to exchange data. This will allow you to modify parameters in your model and see the results on the hardware:



Notice as it is compiles how much larger the program size is (at the bottom of your Simulink diagram click ‘View diagnostics’):

```
Diagnostic Viewer

▼ Top Model Build

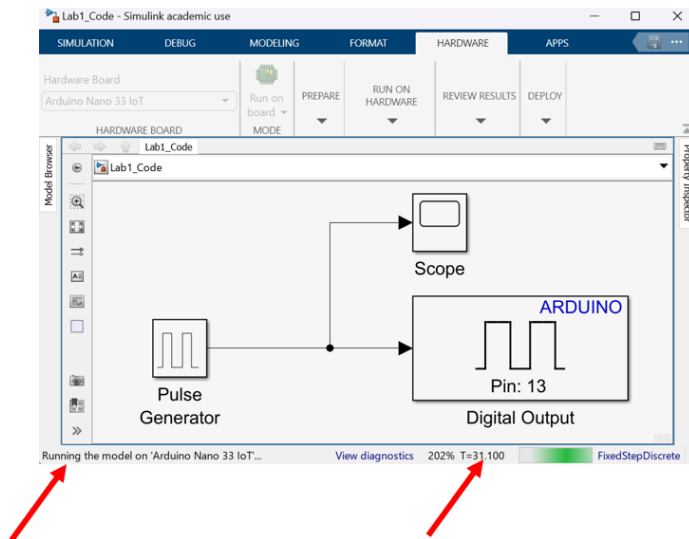
### Starting build procedure for: Lab1_Code
-----
ARM Memory Usage
-----
device - ARM Cortex-M0+

Program: 33112 bytes (12.6% Full)
(.text)

Data: 4604 bytes (14.1% Full)
(.data + .bss)
### Deploying code on Arduino Nano 33 IoT on port COM54
### Deployed code to target successfully
```

Notice that to allow communication with the board it uses twice as much program memory some additional data memory. The interactive communication takes more program memory and could possibly be an issue.

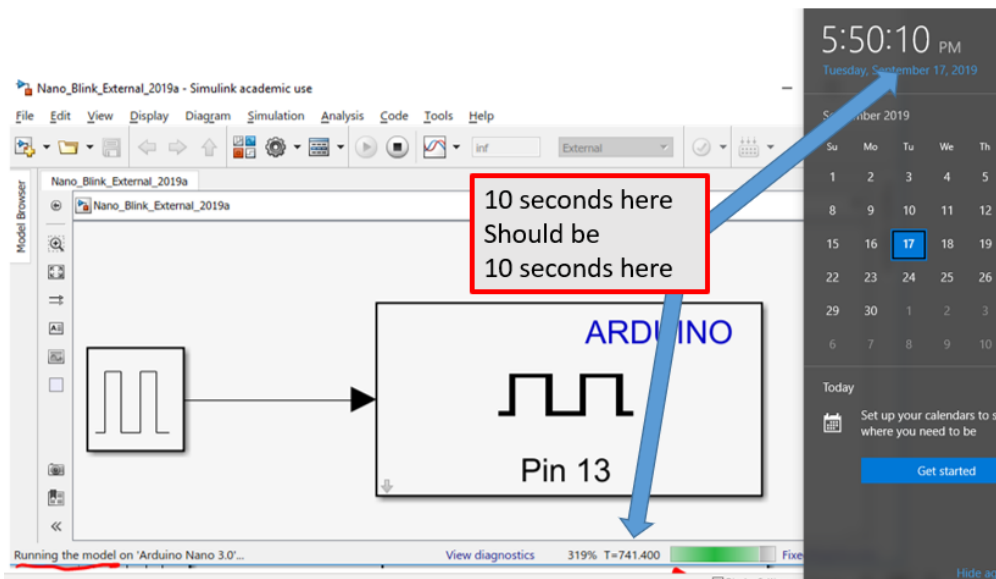
After the code is downloaded it will connect to the hardware and you will see the time progressing in the bottom right corner:



- 2.) Change parameters in your diagram and observe changes on the hardware. Double Click the Pulse Generator block and change the Period to 0.1 seconds. You should notice that the LED will begin blinking much faster.
- 3.) Use this file for new projects! For new hardware projects always use this file – it is configured to work with the hardware and you will not need to perform all the configuration steps again.

External Mode

- **External mode** and the green play button **allows bi-directional communication** to/from the application board to the PC, but can limit how fast your code can execute.
 - The “**Deploy to Hardware Button**” downloads the code and **does not connect or interact with it**, it can allow your code to run faster. You can use the “**Deploy to Hardware Button**” to download the code and NOT communicate to it.
- When external mode is selected and the green play button is pressed it downloads additional code to the board to allow this communication. This **increase** the program size (requires more memory)
- When you use external mode, you can change parameters while the system is running
 - However, this could have an impact on the performance due to the communication bandwidth
 - You can tell how fast you can run your code by examining the running clock in the Simulink diagram after you press the play button. When the code is running the green bar will be flashing and the time in seconds should be advancing:



If this clock advances as fast as the normal real world clock your code is “keeping up”. If, for example, it can take 1 minute for the this timer to reach 10 seconds your code is not keeping up with real time and your requested sample time is to fast – it can not compute everything it needs to do fast enough.

Stopping the Simulation:

When you are running the simulation in external mode you are connected to the device and are sending data and receiving data.

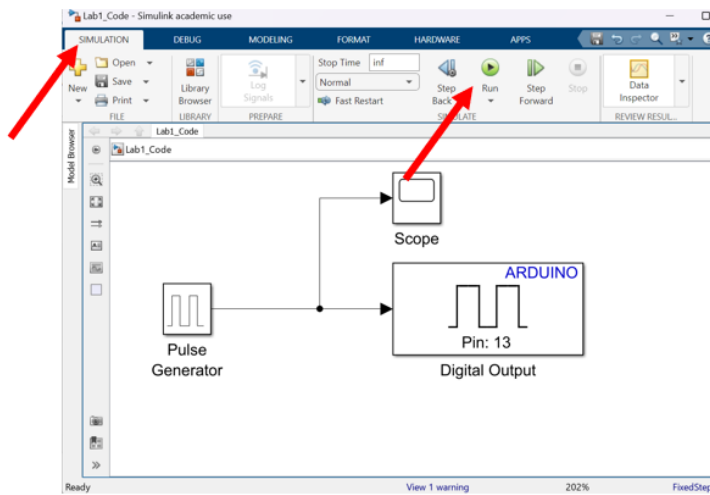
- Use the green “Play” button to download and run the code in external mode
- Use the square “Stop” button to stop.

Checkpoint:

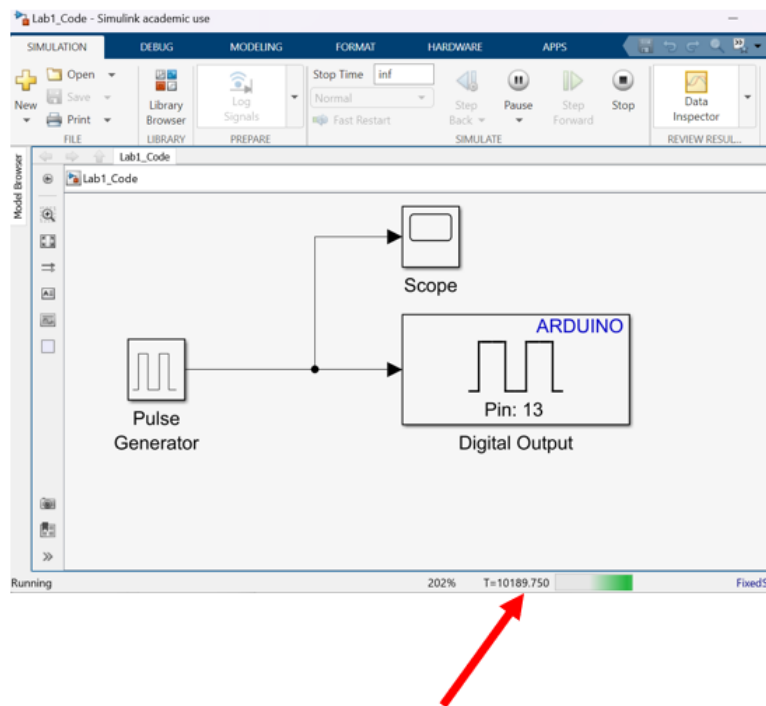
To complete Lab 1 you will need to show the ability to control the blink of the LED by changing the parameters of the pulse block in external mode.

Simulation Mode:

At the top of you Simulink diagram click on the ‘Simulation’ Tab, then press run.



Notice how fast the time is changing:



This indicates thousands of seconds have passed, which is not true. The simulation tab simulates (solves) the diagram, but does nothing with the Hardware. If you click on the scope you can see it is creating the Pulse Generator output, but nothing is happening on the hardware.

Make sure the Hardware tab is selected to run the code on the hardware!

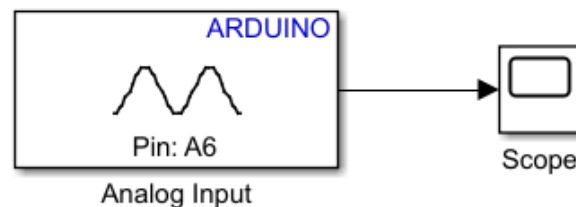
Part 3 : Evaluating code execution speed – how fast can I run my code

Objective:

- Determine how fast your code can be run on the target hardware using “Enable overrun detection” in Simulink.

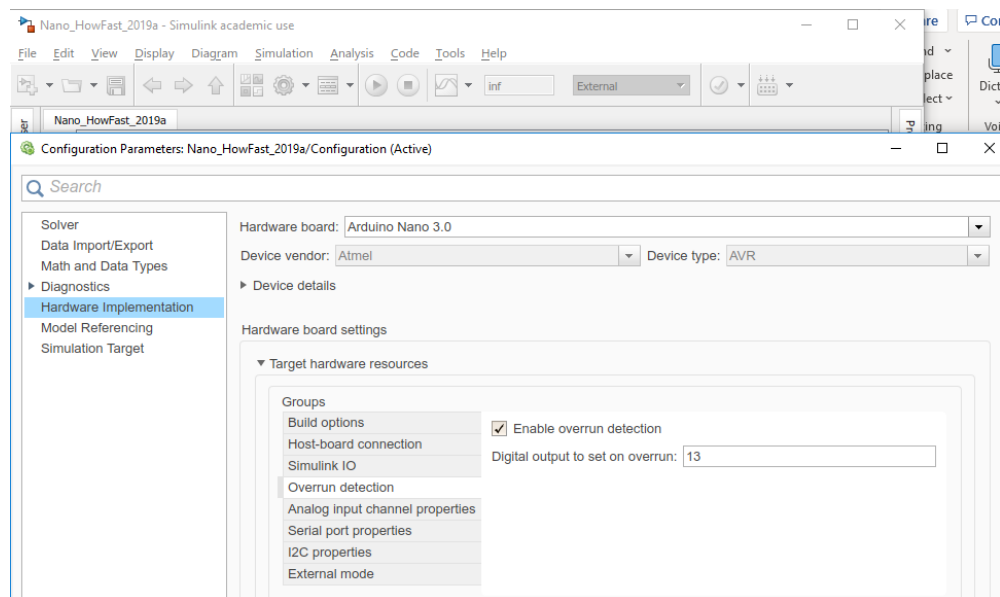
Simulink:

Build the following model:



This code reads an analog pin the hardware and displays the result in the scope. To determine how fast the hardware can execute a simple analog read check the box “Enable overrun detection” and set the pin to 13:

In the “Model Configuration Parameters” under “Hardware Implementation”, “Target hardware resources”, “Overrun detection” make sure “enable overrun detection is checked” and the output pin is set to 13.



If the hardware cannot execute your code in the sample time you specify the LED on pin 13 will light up indicating that you code cannot be executed in the time you have specified. You can use any digital

output, but pin 13 is conveniently connected to an LED on our board. For the example code above, start with a sample time of 0.01 seconds. Download the code with the deploy to hardware button (Deploy to hardware mode, NOT external mode play button) and observe if LED 13 on the board lights up after downloading:

Configuration Parameters: Lab1_Code_Part3_R1/Configuration (Active)

Search

Solver

- Data Import/Export
- Math and Data Types
- Diagnostics
- Hardware Implementation
- Model Referencing
- Simulation Target

Simulation time

Start time: 0.0 Stop time: inf

Solver selection

Type: Fixed-step Solver: discrete (no continuous states)

▼ Solver details

Fixed-step size (fundamental sample time): .01

Zero-crossing options

☐ Enable zero-crossing detection for fixed-step simulation

You should notice LED 13 will not light up – this means it has no issue running the code at 10 milliseconds.

Keep decreasing the sample time (try .008 seconds, etc.) until LED 13 **does** light up after downloading. If it lights up, it means it is not able to perform your computations at that sample time.

Using this procedure try and see how fast you can run the code above to the nearest millisecond.

In this fashion, you can determine how fast your code, or any part of your code takes to run on your hardware.

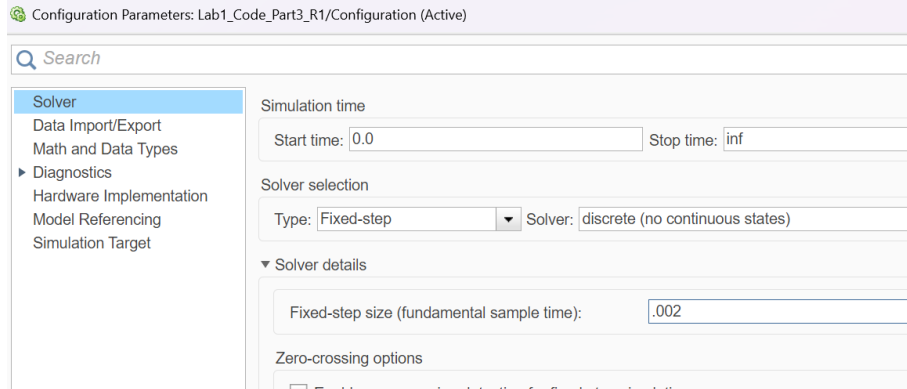
Checkpoint

What is the fastest you can run the code using 'Deploy' (to the nearest millisecond)?

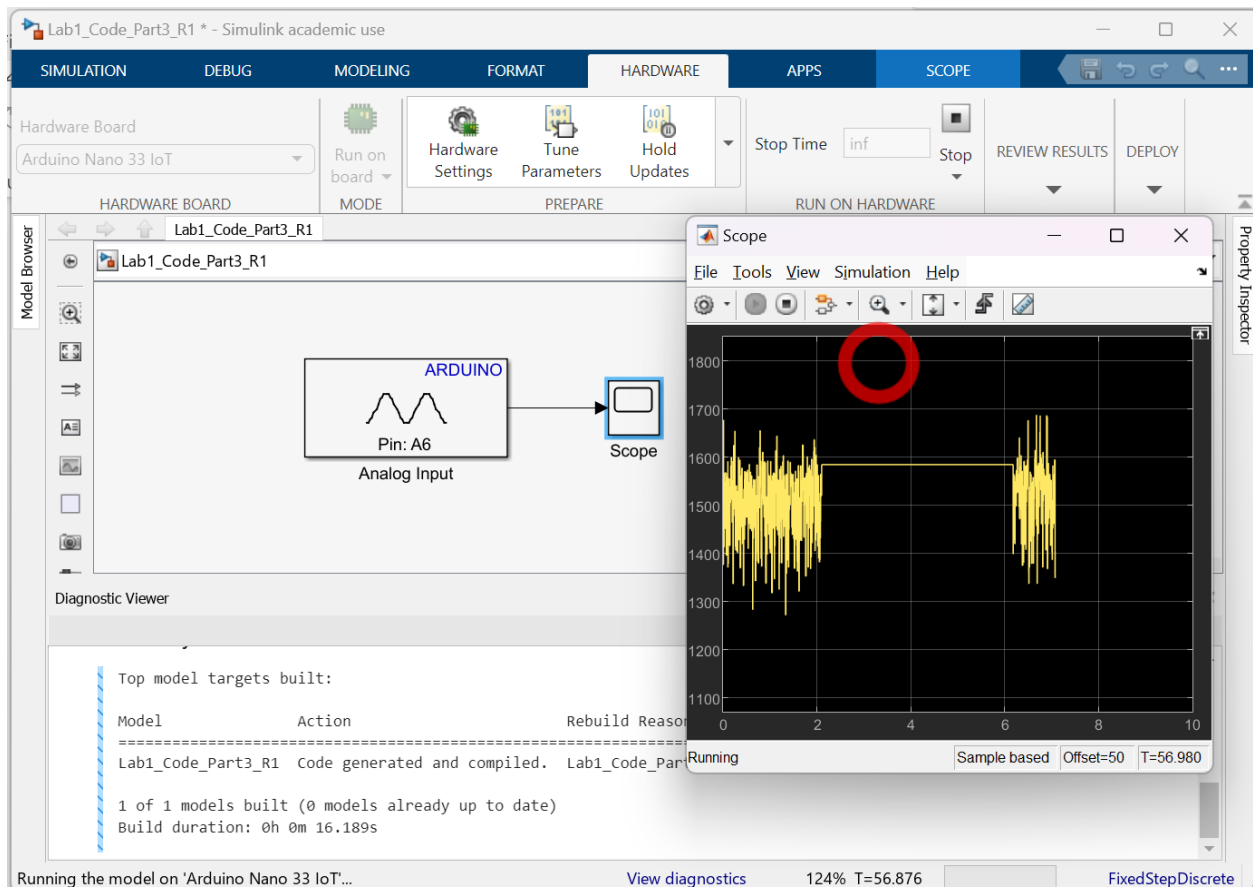
How fast can I run my code in Monitor and Tune?

Now that you have an idea how fast you can run the code on the device, now see how fast you can run the code AND get data from the hardware.

Change the sample time to 0.002 (Two milliseconds)

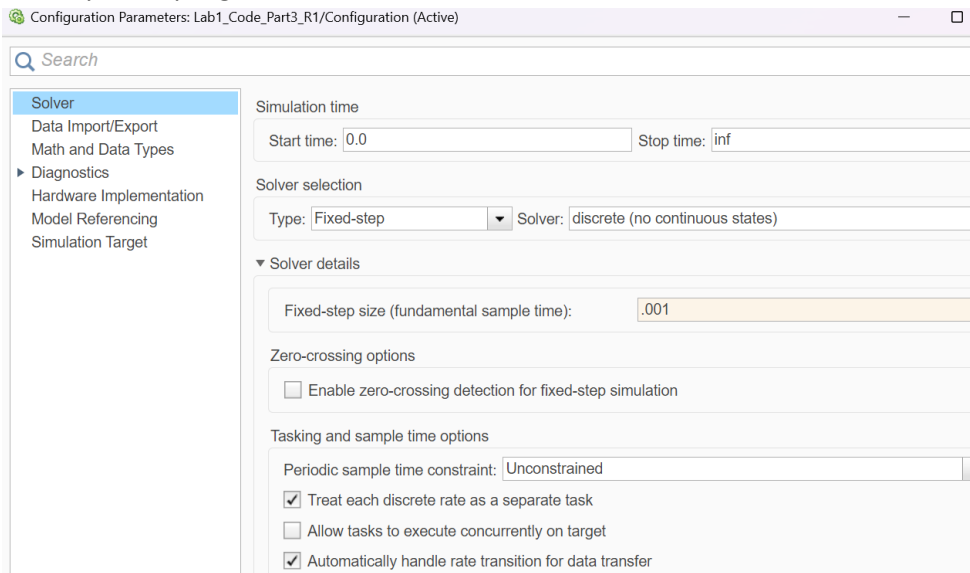


Run the code with 'Monitor and Tune'. The code should run and you should be able to see data in the scope. In addition the clock in the bottom should be advancing in real time:



If you let the code run long enough you may notice occasionally there is a “pause” in the data, or it may disconnect and give you a communication error. This usually is an indication that the sampling time, or communication parameters might be getting close enough to the current limit. And you will need to use a larger sample time.

Now try a sampling time of .001.



Now you should notice the code might run for a short time then just stop, or might not run at all. This means it just cannot communicate reliably enough at this sample time.

Checkpoint

What is the lowest sampling time (nearest millisecond) that you can run in 'Monitor and Tune' and reliability view data for a minute or more?

Part 4: Troubleshooting

An error appears when trying to run the file similar to “build procedure aborted”, “make command returned an error...”

- Verify **Simulink** Support for Arduino Hardware is installed and appears in the Simulink Library browser
- Ensure that the Matlab directory in **your documents**/MATLAB folder or similar location (NOT the programs folder)
- Open MATLAB Before opening the Simulink file (sometimes if you open MATLAB directly with the file it will open in the programs folder instead of your documents folder)

Error appears when trying to open the Simulink file:

- Ensure your downloaded file does not have parenthesis in the file name for example:
 - Bad file name “Lab_0_Step_Response (1).slx” This happens when you download the file several times, it will append (1) to the filename and Matlab files cannot have parenthesis in the names.

Simulink Support for Arduino Hardware does not appear in the Simulink library browser:

- Restart Matlab and Reinstall **Simulink** Support for Arduino Hardware

Device is not recognized in device manager when it is plugged in

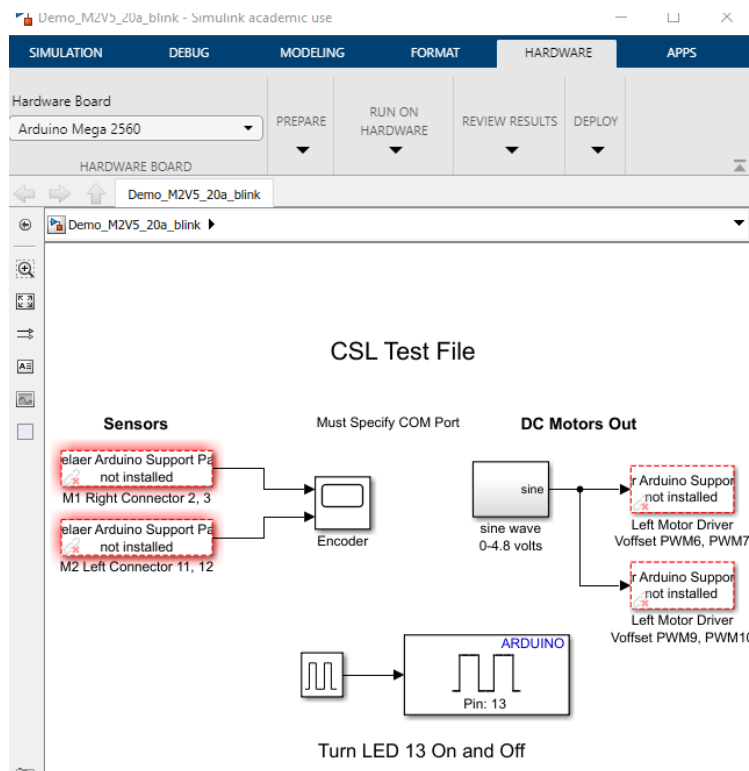
- Open device manager in Windows, find the device, right click and choose Update device drivers or install the drivers via “manage add ons menu”

File is not compiling, a long list of errors appears.

- Verify that RASPLib is installed and that verify Rensselaer Arduino Support Package appears in the file browser.
- Verify that RASPLib folder and startup.m are located in
 - Documents\MATLAB\RASPLib
 - Documents\MATLAB\startup.m

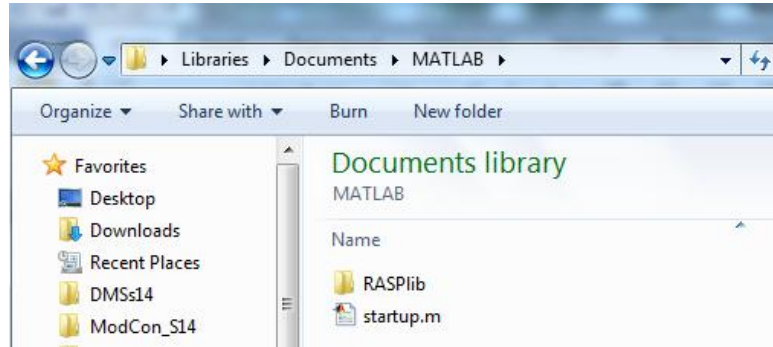
In addition to the above, make sure that MATLAB working directory is not Windows/system32 or some other system folder that requires admin rights to edit. To be safe, set the working folder to your documents/MATLAB

Red block in Simulink diagram:



The Arduino Support package or RASPLib are not properly installed. Follow the pre-lab instruction carefully. The most common error is not placing startup.m and RASPLib in the correct location:

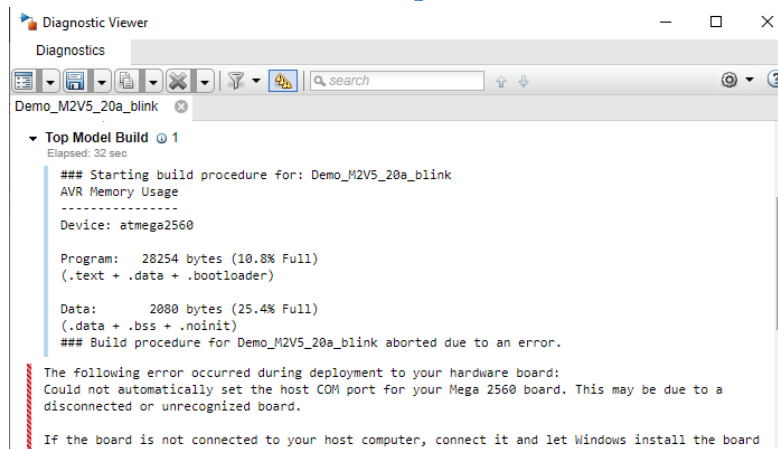
Exactly as shown below:



- **YES** C:\Users\hurstj\Documents\MATLAB\RASPLib
- **YES** C:\Users\hurstj\Documents\MATLAB\startup.m
- **NOT** C:\Users\hurstj\Documents\MATLAB\RASPLib\RASPLib
- **NOT** C:\Users\hurstj\Documents\MATLAB\RASPLib\startup.m
- **NOT** C:\Users\hurstj\Documents\MATLAB\hurstj01-RASPLib-366dd1e\RASPLib

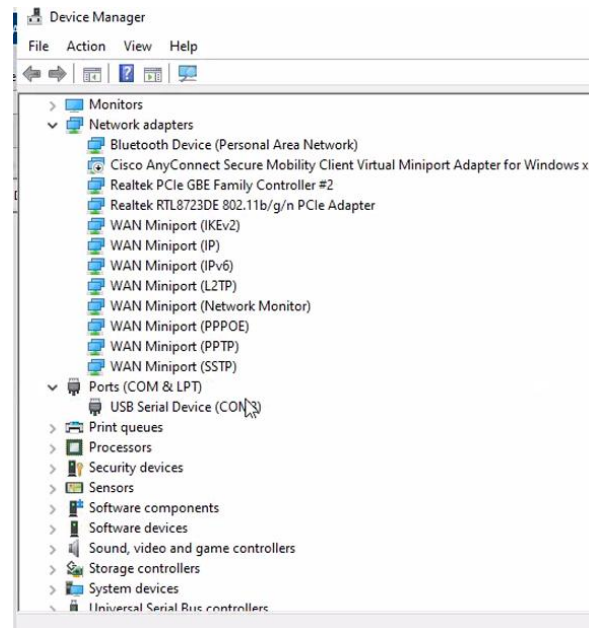
Fix and restart matlab.

Error about board not connected to computer

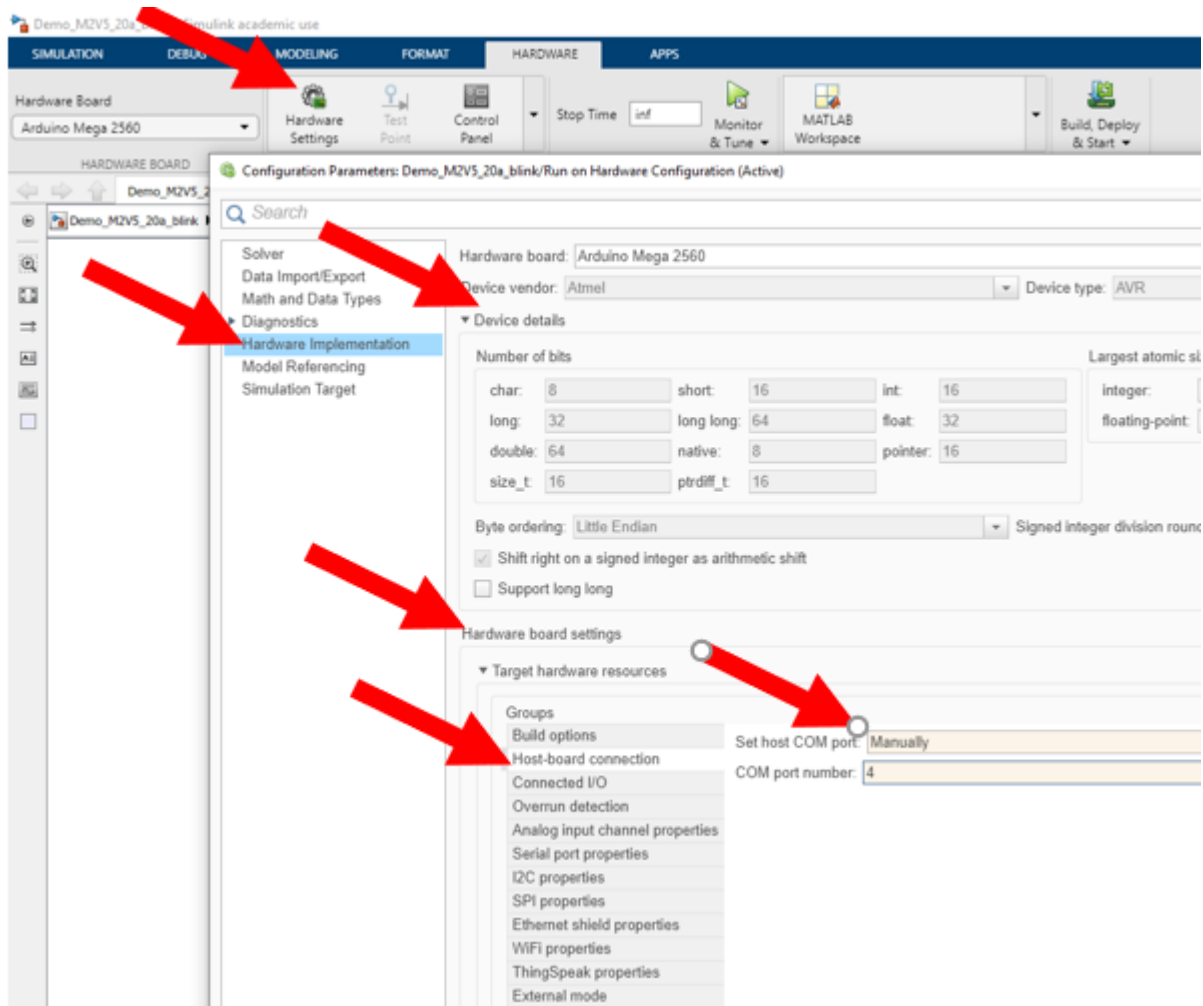


First Check that board is recognized by your computer:

Press windows key, type “device manager”, scroll down to Ports, and expand. Connect the USB cable to the computer and see what device it shows. It should read “USB-SERIAL CH340”. If it does not right click on the device and select “update driver” and search automatically. After this is done disconnect and reconnect the USB cable. Or install the drivers in the CH341SERexe.zip.

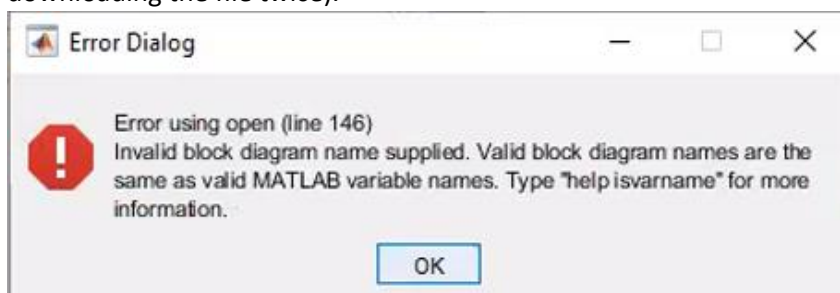


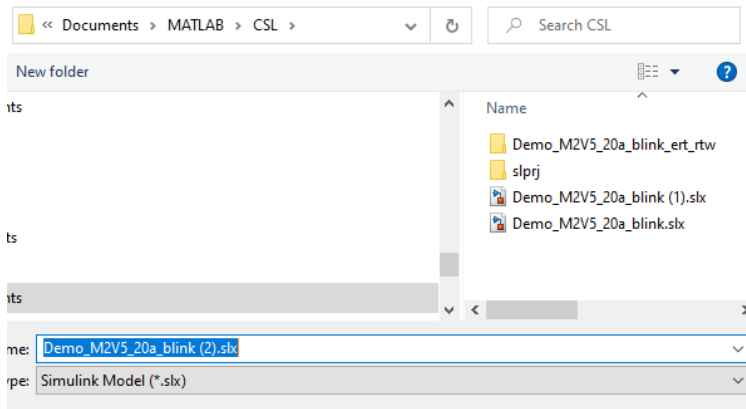
Verify the Simulink diagram is using the correct COM port number. Then in the Simulink diagram click “Hardware Settings”, “Hardware Implementation”, expand the “Device Details”, “Hardware board settings”, “Target hardware Resources”, “Host Board connection”, then “set host Com port” to “Manually” and specify the com port number.



Error Dialog. Error using open... Simulink diagram file name has parenthesis in it

- Make sure the Simulink diagram does not have parenthesis in it (which can be caused by downloading the file twice).





Questions:

How can you determine how fast you are able run code in External mode?

How can you determine how fast you are able to run run in Deploy to hardware (normal) mode?

What is the difference between Deploy to hardware and External mode?

What can happen if the path to your compile folder is too long?

What can happen is there are parenthesis in the file name?

What is the fastest you can run the Analog example using 'Deploy'?

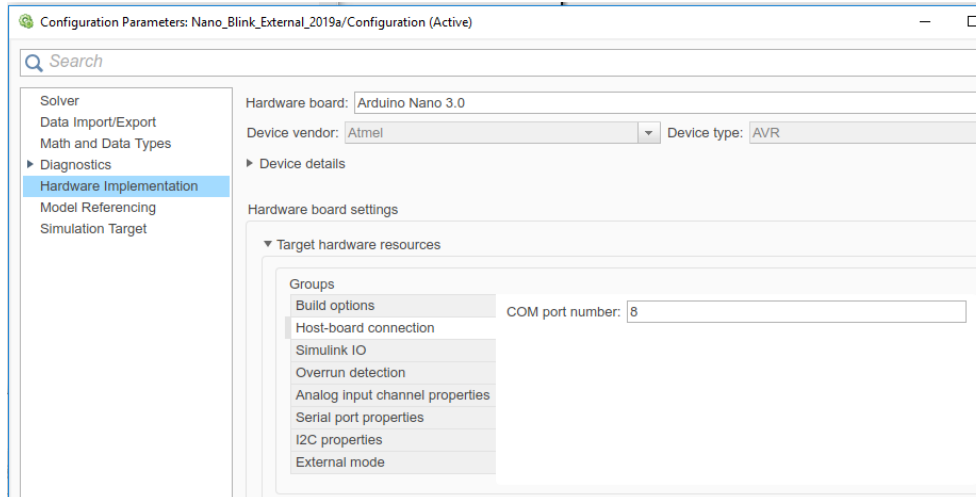
What is the fastest you can reliably view data from the Analog example using 'Monitor and Tune'?

What happens when you use the 'Run' button in the Software tab?

Troubleshooting:

Setting the COM port manually

- Set COM port to the number you found in the device manager by opening “Target hardware resources” and then “Host-board connection”.



- Right click on the background of the Simulink window and select ‘Model Configuration Parameters’, or  in the top of the Simulink window.

Fails to compile in less than 5 seconds with

Model Explorer

File Edit View Tools Add Help

Model Hierarchy

- Simulink Root
 - Base Workspace
 - Demo_MinSegNano33_V3_24a_R5*
 - Lab1_Code_Part3*
 - Configurations
 - Model Workspace
 - External Data

Contents of: ...b1_Code_Part3/Configurations (only) [Filter Contents](#)

Column View: Configurations [Show Details](#) [2 object\(s\)](#)

Name	Description
Configuration (Active)	
Configuration1	

Lab1_Code_P

This section of the model. You can currently operate on the model.

Diagnostic Viewer

```

C:/PROGRA~1/MATLAB/R2024a/toolbox/target/shared/armcortexmbase/xcp/include -D _ert_main_0 -C:/Users/hurstj/
compilation terminated.
gmake[1]: *** [rtiostream_serial_daemon.o] Error 1
gmake[1]: *** Waiting for unfinished jobs....
gmake[1]: Leaving directory `C:/Users/hurstj/Documents/MATLAB/compile24a/Lab1_Code_Part3_ert_rtw'
gmake: *** [all] Error 2
The make command returned an error of 2
### Build procedure for Lab1_Code_Part3 aborted due to an error.
  
```

▼ **Build Summary**

```

Top model targets built:

Model          Action  Rebuild Reason
=====
Lab1_Code_Part3 Failed  Code generation information file does not exist.

0 of 1 models built (0 models already up to date)
Build duration: 0h 0m 4.5304s
  
```

Error(s) encountered while building "[Lab1_Code_Part3](#)"

Component: Simulink | Category: Block diagram error